COLÉGIO COTEMIG – SISTEMA DE PROVAS TÉCNICAS PRÁTICAS

AVALIAÇÃO: TRIMESTRAL – 1º TRIMESTRE – 3ª SÉRIE **TURNO:** MANHÃ - A

DISCIPLINA: PYTHON **VALOR:** 14 pontos

PROFESSORA: JANAÍNA DUARTE **PERÍODO:** 17 A 21/08/2026

Nome:	Nº:	Turma:	NOTA

INSTRUÇÕES:

1. **Leia integralmente o enunciado antes de iniciar a prova. A correta interpretação faz parte dos critérios de avaliação.**
2. **A prova é individual. Cada aluno deve desenvolvê-la utilizando exclusivamente seu próprio login.**
3. **Crie, em seu diretório de trabalho, uma pasta denominada prova_python_(suaMatricula). Todos os scripts desenvolvidos deverão ser armazenados obrigatoriamente nessa pasta.**
4. **Faça o download do conteúdo de sua prova e salve dentro da pasta prova_python_(suaMatricula).**
5. **Crie uma pasta com o nome prints_thunder dentro da pasta prova_python_(suaMatricula).**
6. **Ao final, compacte a pasta prova_python_(suaMatricula em formato .zip e envie o arquivo pela atividade correspondente no Google Classroom.**

COLÉGIO COTEMIG – SISTEMA DE PROVAS TÉCNICAS PRÁTICAS

AVALIAÇÃO:	TRIMESTRAL – 1º TRIMESTRE – 3ª SÉRIE	**TURNO:**	MANHÃ - A
DISCIPLINA:	PYTHON	**VALOR:**	14 pontos
PROFESSORA:	JANAÍNA DUARTE	**PERÍODO:**	17 A 21/08/2026

PROVA PRÁTICA

Você é **programador(a) júnior** recém-contratado(a) pela empresa **The Best Cotemig**.

Na segunda-feira o time de dados entrega o projeto **RaspaLance**: uma API Flask que faz **webscraping** das notícias de futebol do site [lance.com.br](https://www.lance.com.br/), grava coletas no SQLite e mostra um front HTML.

Problema: o estagiário anterior “organizou” o código na sexta à tarde… e apagou pedaços importantes em algumas camadas. O sistema **não sobe**. A CTO ligou e pediu:

> *“Arruma o RaspaLance até o fim do expediente. Preciso do scraping, do histórico, do site e de testes no Thunder Client.”*

Sua missão: **consertar o projeto** e **comprovar a API** no Thunder Client.

Comece por:

pip install -r requirements.txt

python app.py

BOA SORTE

COLÉGIO COTEMIG – SISTEMA DE PROVAS TÉCNICAS PRÁTICAS

AVALIAÇÃO: TRIMESTRAL – 1º TRIMESTRE – 3ª SÉRIE **TURNO:** MANHÃ - A

DISCIPLINA: PYTHON **VALOR:** 14 pontos

PROFESSORA: JANAÍNA DUARTE **PERÍODO:** 17 A 21/08/2026

PARTE 1 — Consertar o código (**10,0 pontos**)

Leia os erros do terminal, abra os arquivos e complete/corrija o que foi apagado.

A pontuação segue as **camadas** afetadas.

Etapa 1.1 — Models (**3,0 pts**)

Arquivos: `models/base.py`, `models/coleta_noticia.py`, `models/__init__.py`

(e confira `models/noticia.py` se necessário)

```
| O que entregar | Pts |
|----------------|-----|
| `ModeloBase` importa e usa `datetime` corretamente; campos de chave/`__abstract__` ok | **1,2** |
| Colunas de `ColetaNoticia` com tipos SQLAlchemy válidos (ex.: `String`) | **1,0** |
| `models/__init__.py` exporta tudo que a aplicação precisa (`db`, models…) | **0,8** |

**Dica:** sem models corretos, o Flask-SQLAlchemy não cria as tabelas do histórico.
```

--### Etapa 1.2 — Controllers (API JSON) (**3,0 pts**)

Arquivos: `controllers/noticias_api.py`, `controllers/historico_api.py`, `controllers/__init__.py`

```
| O que entregar | Pts |
|----------------|-----|
| Imports do Flask corretos (`Blueprint`, etc.) na API de notícias | **1,0** |
| Rotas com **método HTTP** certo (GET listar / POST sincronizar) | **1,0** |
| Blueprint do histórico criado e registrado; listagem de coletas com método GET | **1,0** |
```

Dica: `methods=[""]` ou `methods=[]` não é rota válida.

Etapa 1.

3 — Controllers (site HTML) + Views (**2,5 pts**)

Arquivos: `controllers/site_controller.py`, `app.py`, pasta `views/`

```
| O que entregar | Pts |
|----------------|-----|
| Decorators `@` nas rotas do site; formulário `/buscar` aceita **POST** | **1,0** |
| `render_template` do detalhe da coleta recebe a variável `coleta` completa | **0,7** |
| Pasta de templates bate com o `template_folder` do `app.py` (renomear pasta **ou** ajustar o caminho) | **0,8** |

**Dica:** a pasta pode estar como `views/template` (singular) e o `app.py` pedir `views/templates` (plural). Escolha **uma** correção e seja consistente.

---
```

Etapa 1.4 — Integração: sobe e raspa (**1,5 pts**)

```
| O que entregar | Pts |
|----------------|-----|
| `python app.py` sobe sem traceback | **0,5** |
| Site em `http://127.0.0.1:5000` abre e o botão **Raspar notícias** mostra tabela | **0,5** |
| `GET /api/noticias?modo=futebol` devolve JSON com `"total"` > 0 (com internet) | **0,5** |

---
```

PARTE 2 — Thunder Client (**4,0 pontos**)

Com a Parte 1 funcionando, use o **Thunder Client** (extensão do VS Code / Cursor).

Etapa 2.1 — Instalar o Thunder Client (**1,0 pt**)

```
### Etapa 2.1 – Instalar o Thunder Client (**1,0 pt**)

| O que entregar | Pts |
|----------------|-----|
| Extensão Thunder Client instalada e aberta | **1,0** |

Print ou demonstração para o professor.
```

Etapa 2.2 — Consultar o backend (GET) (**1,5 pts**)

GET não usa body.

Configure só o método, a URL e (se quiser) a query string.

| O que entregar | Pts |

GET executado com sucesso (status **200**) + print da resposta JSON | **1,5** |
ATENÇÃO PRINT O STATUS

Etapa 2.3 — Postar algo no backend (POST) (**1,5 pts**)

Faça **um** dos POSTs abaixo. Em ambos: Method = `POST`.

cadastro **manual** (recomendado) — body JSON

Method	`POST`
URL	`http://127.0.0.1:5000/api/noticias/manual`
Header	`Content-Type` = `application/json`
Body	tipo **JSON**, com o conteúdo abaixo

```json
{
  "titulo": "The Best Cotemig vence a final do interclasse",
  "url": "https://www.lance.com.br/exemplo-prova.html",
  "secao": "manual"
}
```

Resposta esperada: status **201**, com `coleta_id` e a notícia gravada.

Confirme depois com `GET /api/historico/coletas` ou `GET /api/historico/coletas/<id>` (**sem body**).

| O que entregar | Pts |

| POST executado com sucesso (**201**) + evidência (print / resposta) | **1,5** |